

Platform Amendment — Version 4.4.0

Vermont Health Platform (HTR)

Type: Amendment to Platform Documentation (supplements v4.3.0 amendment and printed v4.2.0 docs) **Version:** 4.4.0 **Date:** April 2026
Classification: Internal **Scope:** All new features, architectural changes, security improvements, and database migrations implemented after the v4.3.0 amendment. This document covers ten major feature areas implemented as the second half of the Phase 2/3 improvement plan.

Table of Contents

- 1. [Summary of Changes](#)
- 2. [HTI Dashboard — Real Data Wiring](#)
- 3. [API Key Rotation](#)
- 4. [Incremental RAG Indexing](#)
- 5. [Sanity Webhook — Incremental Triggers](#)
- 6. [Embedding Cache](#)
- 7. [Streaming Structured Output](#)
- 8. [OpenAI Fallback Chain](#)
- 9. [HTR Connect Hub](#)
- 10. [Professional Member Directory](#)
- 11. [Research Lab — Bundle Optimization & Error Boundaries](#)
- 12. [Admin Revenue Dashboard](#)
- 13. [Hospital Real Data Pipeline](#)
- 14. [Database Migration 023](#)
- 15. [Environment Variables — New Additions](#)
- 16. [Updated Known Issues Register](#)

1. Summary of Changes

#	Area	Change	Primary Files
1	Frontend / Backend	HTI Dashboard wired to real CMS/ONC data via Supabase	<code>components/HTIDashboard.tsx</code> , <code>app/api/hti-scores/route.ts</code>
2	Frontend	API key rotation — generate new key while old expires in 24h	<code>app/api/keys/rotate/route.ts</code> , <code>app/account/api-keys/ApiKeysClient.tsx</code>
3	Backend	Incremental RAG reindex — single Sanity document, no full rebuild	<code>backend/services/indexing.py</code>
4	Frontend + Backend	Sanity GROQ webhook — triggers incremental reindex on content change	<code>app/api/webhooks/sanity/route.ts</code> , <code>backend/routers/ingest.py</code>
5	Backend	Embedding cache — skip re-embedding unchanged PDF files	<code>backend/services/indexing.py</code>
6	Backend	Streaming structured output — format detection injects Markdown instructions	<code>backend/routers/chat.py</code>
7	Backend	OpenAI GPT-4o-mini as last-resort fallback when Groq is fully down	<code>backend/services/llm.py</code>
8	Frontend	HTR Connect hub — landing page for all community features	<code>app/connect/page.tsx</code> , <code>components/HomeSidebar.tsx</code>
9	Frontend + Backend	Professional member directory — opt-in profiles, search, edit	<code>app/connect/directory/</code> , <code>app/api/directory/route.ts</code>
10	Frontend	Research Lab — all 19 tools lazy-loaded with skeletons and error boundaries	<code>app/research-lab/ResearchLabHub.tsx</code> , <code>components/ErrorBoundary.tsx</code>
11	Frontend	Admin Revenue Dashboard — MRR, churn, trial conversion, plan breakdown	<code>app/admin/revenue/page.tsx</code>
12	Backend + Frontend	CMS hospital data pipeline — real data replaces synthetic hospital view	<code>backend/scripts/load_cms_hospitals.py</code> , <code>app/dashboard/[state]/page.tsx</code>
13	Database	Migration 023 — API key rotation schema + professional_profiles table	<code>supabase/migrations/023_api_key_rotation_and_directory.sql</code>

2. HTI Dashboard — Real Data Wiring

Background

The Health Transformation Index (HTI) Dashboard displayed synthetic/mock data only. This change wires it to real data sourced from CMS Quality Payment Program API and ONC Health IT Dashboard, stored in the `hti_scores` Supabase table (created in Migration 022).

API Route

frontend/app/api/hti-scores/route.ts (New)

A lightweight GET endpoint that reads from hti_scores with optional filtering:

```
GET /api/hti-scores?state=vermont&quarter=2024-Q3

Response:
{
  scores: HtiScoreRow[],
  source: "supabase" | "empty"
}
```

- Uses the anon Supabase client with cookie context for SSR compatibility
- Supports filtering by state_id and quarter via URL query parameters
- Returns empty array gracefully if no data exists — does not throw
- Cache TTL: 1 hour (revalidate = 3600) — HTI scores are updated quarterly

HtiScoreRow shape (matches hti_scores table from Migration 022):

```
{
  state_id:      string;    // e.g. "vermont"
  quarter:      string;    // e.g. "2024-Q3"
  composite:     number;    // 0–100
  policy:       number;
  economics:    number;
  technology:   number;
  clinical:     number;
  equity:       number;
  source:       string;    // "CMS QPP" | "ONC" | "HTR"
}
```

HTI Dashboard Component Changes

frontend/components/HTIDashboard.tsx (Modified)

Real data fetch — on mount, the component fetches live scores and groups them by state:

```
const [realScores, setRealScores] = useState<Record<string, QuarterlySnapshot[]>>({});

useEffect(() => {
  fetch("/api/hti-scores")
    .then(r => r.json())
    .then(json => {
      const byState: Record<string, QuarterlySnapshot[]> = {};
      (json.scores ?? []).forEach((s: HtiScoreRow) => {
        if (!byState[s.state_id]) byState[s.state_id] = [];
        byState[s.state_id].push(s);
      });
      setRealScores(byState);
    })
    .catch(() => {}); // Silent – mock data is still displayed if fetch fails
}, []);
```

Overlay dataset — the trend chart useMemo appends a second dataset when real data exists for the selected state:

```
...(stateRealSnapshots.length > 0 ? [{
  label: "Real Data (CMS/ONC)",
  data: allLabels.map(q => {
    const snap = stateRealSnapshots.find(s => s.quarter === q);
    return snap ? (trendDomain === "composite" ? snap.composite : snap[trendDomain]) : null;
  }),
  borderColor: "#F59E0B", // Amber – distinct from the synthetic blue line
  borderDash: [3, 3],     // Dashed – visually indicates real vs. projected
  fill: false,
  tension: 0.3,
}] : [])
```

Real data badge — a small indicator appears in the chart header when the selected state has real data:

```
{realScores[trendState]?.length > 0 && (
  <span className="text-[10px] font-bold bg-amber-50 text-amber-700 border border-amber-200 px-1.5 py-0.5 rounded-full">
```

```
      ✓ Real Data
    </span>
  )}
```

Loading Real Data (Tutorial)

To populate the `hti_scores` table with real CMS QPP data:

- 1. **Download CMS QPP participation data** from `data.cms.gov/api/1/datastore/sql` — filter by `year`, `state`, and APM participation flags.
- 2. **Normalize to HTI score format:** map QPP APM penetration rate (0–100) to the `economics` pillar. ONC interoperability data maps to `technology`.
- 3. **Upsert via admin script:**

```
cd backend
python -m scripts.load_hti_scores --year 2024 --source "CMS QPP"
```

- 4. The dashboard will automatically display the amber overlay on next page load.

3. API Key Rotation

Background

Before this change, API keys were permanent — if a key was compromised, the only option was revocation, which immediately broke integrations. Key rotation provides a grace period: generate a new key, old key expires in 24 hours, integrations have time to update.

New API Route

`frontend/app/api/keys/rotate/route.ts` (New)

```
POST /api/keys/rotate
Authorization: session cookie (same as other account routes)
Body: { keyId: string }

Response:
{
  key: string,           // Raw new key – shown once, never stored
  record: ApiKeyRecord,  // New DB record
  oldKeyExpiresAt: string // ISO timestamp 24h from now
}
```

Rotation algorithm (mirrors the create flow):

```
// 1. Verify ownership and active status
const { data: oldKey } = await dbAdmin
  .from("api_keys")
  .select("id, user_id, name, tier, revoked_at")
  .eq("id", keyId)
  .eq("user_id", user.id)
  .single();

if (!oldKey || oldKey.revoked_at) throw 400;

// 2. Generate new key with identical algorithm as /api/keys/create
const random    = crypto.getRandomValues(new Uint8Array(16));
const randomHex = Array.from(random).map(b => b.toString(16).padStart(2, "0")).join("");
const hmac      = await crypto.subtle.sign("HMAC", hmacKey, payload);
const rawKey    = `htr_${randomHex}${hmacHex.slice(0, 16)}`;

// 3. Insert new record, linking to old key
const { data: newRecord } = await dbAdmin.from("api_keys").insert({
  user_id:    user.id,
  name:       oldKey.name,
  tier:        oldKey.tier,
  key_prefix: rawKey.slice(0, 12),
  key_hash:   sha256(rawKey),
  rotated_from: oldKey.id, // ← FK to predecessor
}).select().single();

// 4. Set 24-hour expiry on old key
await dbAdmin.from("api_keys")
  .update({ expires_at: expiresAt })
  .eq("id", oldKey.id);
```

Why 24 hours? This provides enough time for most CI/CD pipelines and automated jobs to pick up the new key from secrets management (e.g., GitHub Actions secrets, Railway variables). The old key continues working until `expires_at` is reached, at which point the `revoke_expired_api_keys()` pg_cron function automatically sets `revoked_at`.

UI Changes

`frontend/app/account/api-keys/ApiKeysClient.tsx` (Modified)

- Added `rotating: string | null` state — tracks which key ID is in-flight
- Added **"Rotate"** amber button beside each key's "Revoke" button
- On success: the old key row is replaced with the new record, and `justCreated` state triggers the raw-key display banner (same emerald success banner used for new keys)
- The banner includes messaging about the 24-hour grace period on the old key

User flow:

```
Account → API Keys
  [Key name] [htr_a1b2c3...]  [Rotate] [Revoke]
    ↓ click Rotate
→ Confirm dialog: "Rotate this key? Old key expires in 24 hours."
  ↓ confirm
→ POST /api/keys/rotate
  ↓
→ Success banner: "New key created. Your old key expires in 24 hours.
                  Copy it now – it won't be shown again."
→ [New key displayed with copy button]
```

How Expired Keys Are Cleaned Up

The `revoke_expired_api_keys()` function (defined in Migration 023, scheduled via pg_cron) runs hourly:

```
UPDATE api_keys
SET     revoked_at = NOW()
WHERE   expires_at < NOW()
AND     revoked_at IS NULL;
```

This means an expired key becomes revoked at most 1 hour after its `expires_at` timestamp. The middleware that validates API keys checks both `revoked_at IS NULL` AND `expires_at > NOW()`, so expired keys stop working immediately at `expires_at` regardless of the cleanup job.

4. Incremental RAG Indexing

Background

Before this change, every content update in Sanity CMS required a full index rebuild — fetching and re-embedding all documents (typically 15–20 minutes). This change enables single-document updates in ~30 seconds.

Architecture

```
Sanity CMS content change
|
▼ (Sanity webhook fires)
POST /api/webhooks/sanity (Next.js proxy)
|   Forwards with X-Sanity-Signature header
|
▼
POST /api/ingest/webhook (Python backend)
|   Verifies HMAC-SHA1 signature
|   Extracts _id of changed document
|
▼ (if index is loaded)
_run_incremental(doc_id, index) [background task]
|
▼
incremental_reindex_document(doc_id, index)
|   1. DELETE existing chunks from rag_documents (by doc_id metadata)
|   2. Fetch updated document from Sanity
|   3. Parse into nodes
|   4. Insert new nodes into live index
|
▼ (if index not yet loaded)
_run_job(uuid) [full rebuild as fallback]
```

Implementation

`backend/services/indexing.py` (Modified)

fetch_single_sanity_doc(doc_id) — fetches a single Sanity document by its `_id` using a generic GROQ query that handles all supported content types:

```
_GENERIC_DOC_QUERY = """*[_id == $doc_id][0]{
  _id, _type, title, term, pillar, pillars, summary, description, abstract,
  "bodyText": pt::text(body)
}"""
```

The query extracts text from the Portable Text `body` field using Sanity's `pt::text()` function. Documents without a `body` field (e.g., definitions that only have `description`) still return usable text.

incremental_reindex_document(doc_id, index) — full pipeline:

```
async def incremental_reindex_document(doc_id: str, index: VectorStoreIndex) -> bool:
    # Step 1: Delete existing chunks
    # Supabase REST DELETE on rag_documents where metadata->>'doc_id' = doc_id
    supabase = get_supabase()
    if supabase:
        supabase.table("rag_documents")\
            .delete()\
            .eq("metadata->>doc_id", doc_id)\
            .execute()

    # Step 2: Fetch updated content
    doc_data = await fetch_single_sanity_doc(doc_id)
    if not doc_data:
        log.info(f"Doc {doc_id} not found in Sanity - deleted. Chunks removed.")
        return True # Successfully handled deletion

    # Step 3: Build LlamaIndex Document
    text = doc_data.get("bodyText") or doc_data.get("summary") or doc_data.get("description") or ""
    title = doc_data.get("title") or doc_data.get("term") or "Untitled"
    metadata = {
        "doc_id": doc_id,
        "title": title,
        "pillar": doc_data.get("pillar") or (doc_data.get("pillars") or [""])[0],
        "source_type": doc_data.get("_type", ""),
    }
    document = Document(text=text, metadata=metadata)

    # Step 4: Parse into sentence-window nodes
    parser = SentenceWindowNodeParser.from_defaults(window_size=3)
    nodes = parser.get_nodes_from_documents([document])

    # Step 5: Insert into live index (no rebuild needed)
    index.insert_nodes(nodes)
    log.info(f"✅ Incremental reindex: {doc_id} - {len(nodes)} nodes inserted")
    return True
```

Key design decisions:

- **Delete before insert** — avoids duplicate chunks if a document is updated multiple times. The delete is by `metadata->>'doc_id'` using Supabase's JSON path operator.
- **Deletion is handled gracefully** — if Sanity returns no document (it was deleted), only the deletion step runs. The index now correctly reflects the document's absence.
- **Uses the live index in-memory** — `index.insert_nodes()` writes to both the in-memory `VectorStoreIndex` and the backing Supabase pgvector store simultaneously. No restart needed.
- **Silent fallback** — if the index is not yet loaded (e.g., backend just restarted), the webhook handler queues a full rebuild instead.

Latency Comparison

Operation	Before	After
Full index rebuild	15–20 min	Still available, now triggered only on demand
Single document update	15–20 min (full rebuild)	~30 seconds
Webhook response time	N/A	< 200ms (background task)

5. Sanity Webhook — Incremental Triggers

Architecture

Two components work together: a Next.js forwarding proxy (stateless, always-on) and a Python backend handler.

Next.js Proxy

frontend/app/api/webhooks/sanity/route.ts (New)

Why a proxy? The Python backend on Railway may be sleeping (cold start) or temporarily unreachable. The Next.js proxy on Vercel is always available, so Sanity can always send webhooks without timeout failures. The proxy:

1. Receives Sanity's POST with the `X-Sanity-Signature: sha1=<hmac>` header
2. Forwards the raw body and signature to `${BACKEND_URL}/api/ingest/webhook`
3. Returns HTTP 200 regardless of backend response — this prevents Sanity from entering retry loops during backend downtime

```
export async function POST(request: Request) {
  const body      = await request.text();
  const signature = request.headers.get("X-Sanity-Signature") ?? "";

  try {
    await fetch(`${process.env.BACKEND_URL}/api/ingest/webhook`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "X-Sanity-Signature": signature,
      },
      body,
      signal: AbortSignal.timeout(10_000),
    });
  } catch {
    // Backend offline – swallow error, Sanity gets 200
  }

  return new Response(JSON.stringify({ ok: true }), { status: 200 });
}
```

Python Backend Handler

`backend/routers/ingest.py` — new endpoint `POST /api/ingest/webhook`:

Signature verification — Sanity signs webhooks with HMAC-SHA1 using the shared `INGEST_SECRET`:

```
if INGEST_SECRET:
    sig_header = request.headers.get("X-Sanity-Signature", "")
    body       = await request.body()
    mac        = hmac.new(INGEST_SECRET.encode(), body, hashlib.sha1)
    expected   = "sha1=" + mac.hexdigest()
    if not hmac.compare_digest(expected, sig_header):
        raise HTTPException(status_code=401, detail="Invalid webhook signature")
```

Note: `hmac.compare_digest` is used (not `==`) to prevent timing attacks — a constant-time comparison ensures an attacker cannot determine how many characters of the signature are correct by measuring response time differences.

Document routing:

```
doc_id = payload.get("_id", "")

# Ignore Sanity system documents (e.g. _type starting with "sanity.")
if not doc_id or doc_id.startswith("_"):
    return {"status": "ignored", "reason": "system document"}

index = get_index()
if index is None:
    # Index not loaded – queue full rebuild
    asyncio.create_task(_run_job(str(uuid.uuid4())))
    return {"status": "queued_full_rebuild", "doc_id": doc_id}

# Index is ready – incremental update in background
asyncio.create_task(_run_incremental(doc_id, index))
return {"status": "accepted", "doc_id": doc_id, "mode": "incremental"}
```

Sanity Webhook Configuration

In Sanity Studio → Settings → API → Webhooks:

Setting	Value
URL	<code>https://<your-domain>/api/webhooks/sanity</code>
Secret	Same value as <code>INGEST_SECRET</code> in backend <code>.env</code>
HTTP method	POST
Trigger on	Create, Update, Delete

Setting	Value
Filter	<code>_type in ["policyAnalysis","post","academyModule","caseStudy","definition","analystNote","webinar","report"]</code>
Projections	Leave empty (full document)

The filter prevents webhook floods from drafts (`_id` starting with `drafts.`), settings documents, and image assets.

6. Embedding Cache

Background

Every full index rebuild re-embedded all PDF documents, even if their content had not changed. For large PDF collections, this wasted OpenAI embedding API credits and added minutes to rebuild time.

Implementation

`backend/services/indexing.py` (Modified)

Cache storage: `backend/storage/embedding_cache.json`

```
{
  "<sha256-hash-of-pdf>": [
    { "text": "chunk text...", "metadata": { "file_name": "...", "doc_id": "..." } },
    ...
  ]
}
```

Cache key: SHA-256 hash of the PDF file's binary content. If the file changes (even one byte), the hash changes and the cache entry is invalidated automatically.

Cache functions:

```
def _file_sha256(path: Path) -> str:
    """Stream the file in 64KB chunks to avoid loading large PDFs into memory."""
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(65536), b''):
            h.update(chunk)
    return h.hexdigest()

def _load_embed_cache() -> dict:
    if _EMBED_CACHE_PATH.exists():
        try:
            return json.loads(_EMBED_CACHE_PATH.read_text())
        except Exception:
            return {}
    return {}

def _save_embed_cache(cache: dict) -> None:
    try:
        _EMBED_CACHE_PATH.write_text(json.dumps(cache, indent=2))
    except Exception as e:
        log.warning(f"Could not save embedding cache: {e}")
```

Integration in `build_index()`:

```
embed_cache = _load_embed_cache()
cache_updated = False

for pdf_path in DATA_DIR.glob("*.pdf"):
    file_hash = _file_sha256(pdf_path)
    if file_hash in embed_cache:
        # Reconstruct LlamaIndex Documents from cache
        docs = [Document(text=d["text"], metadata=d["metadata"])
                  for d in embed_cache[file_hash]]
        log.info(f"Cache HIT: {pdf_path.name} ({len(docs)} chunks)")
    else:
        # Load PDF and embed as normal
        docs = load_pdf(pdf_path)
        embed_cache[file_hash] = [{"text": d.text, "metadata": d.metadata} for d in docs]
        cache_updated = True
        log.info(f"Cache MISS: {pdf_path.name} - embedding {len(docs)} chunks")

if cache_updated:
    _save_embed_cache(embed_cache)
```

Impact

Scenario	Without Cache	With Cache
First build (no cache)	Same	Same (cache created)
Rebuild, no PDF changes	Re-embeds all PDFs	Skips all PDFs
Rebuild, one PDF changed	Re-embeds all PDFs	Re-embeds only changed PDF
Estimated saving (10 PDFs, 50 pages each)	~\$0.15 per rebuild	~\$0.00 per rebuild (cached)

The cache persists across backend restarts on Railway because it is stored in the filesystem (`backend/storage/`). On Railway, this directory survives deployments if using a persistent volume; otherwise the cache is rebuilt on each deploy (still correct — just not cached).

7. Streaming Structured Output

Background

The AI Analyst returned free-form text regardless of what the user asked. Requests for comparisons, tables, or step-by-step lists were answered in unstructured paragraphs that rendered poorly in Markdown.

Implementation

`backend/routers/chat.py` (Modified)

Intent detection:

```
def _detect_structured_intent(message: str) -> Optional[str]:
    """
    Returns 'comparison', 'list', or None.
    Case-insensitive. Uses regex for dotted patterns (step.by.step).
    """
    lower = message.lower()
    comparison_phrases = ("compare", "versus", " vs ", "contrast", "difference between",
                          "which is better", "pros and cons", "trade-off", "tradeoff")
    list_phrases = ("table", "list ", "bullet", "summarize", "step by step",
                   "enumerate", "break down", "what are the", "key points")

    if any(p in lower for p in comparison_phrases) or re.search(r"vs\.\?s", lower):
        return "comparison"
    if any(p in lower for p in list_phrases) or re.search(r"step.by.step", lower):
        return "list"
    return None
```

Format instruction injection:

```
structured_intent = _detect_structured_intent(request.message)

if structured_intent == "comparison":
    system_prompt += (
        "\n\nFORMAT INSTRUCTION: The user is requesting a comparison. "
        "Use a Markdown comparison table with columns for each option and rows for "
        "each dimension. Follow the table with a brief narrative summary."
    )
elif structured_intent == "list":
    system_prompt += (
        "\n\nFORMAT INSTRUCTION: The user is requesting a structured list. "
        "Use Markdown headers (##), bullet lists (- item), and tables where appropriate. "
        "Organize information with visible hierarchy."
    )
```

The format instruction is appended to the system prompt, not the user message — this keeps the instruction invisible to the user while directing the model's output format.

Examples of triggered phrases:

User says	Detected intent	Format
"Compare fee-for-service vs value-based care"	comparison	Markdown table
"What are the pros and cons of telehealth?"	comparison	Markdown table
"List the key FHIR R4 resource types"	list	Bullet list
"Step by step, how does ACO attribution work?"	list	Numbered + headers
"What is an ACO?"	None	Free-form prose

8. OpenAI Fallback Chain

Background

The v4.3.0 amendment introduced `FallbackLLM` with Groq→Groq fallback. If Groq itself was fully down (outage, not just rate limit), all AI Analyst functionality failed. This change adds OpenAI `gpt-4o-mini` as a last-resort fallback across all tiers.

Updated Model Routing

`backend/services/llm.py` (Modified)

```
from llama_index.llms.openai import OpenAI as OpenAILLM

def get_llm_for_role(role: str) -> LLM:
    fast_llm = GroqLLM(model=MODEL_FREE, api_key=GROQ_API_KEY)
    sub_llm = GroqLLM(model=MODEL_SUBSCRIBER, api_key=GROQ_API_KEY)

    # Build OpenAI last-resort fallback (only when OPENAI_API_KEY is set)
    openai_fallback: list[LLM] = []
    if OPENAI_API_KEY:
        try:
            openai_fallback = [OpenAILLM(model="gpt-4o-mini", api_key=OPENAI_API_KEY)]
        except Exception:
            pass # llama_index.llms.openai not installed - skip silently

    if role in ("advisory", "admin"):
        if _ANTHROPIC_AVAILABLE and ANTHROPIC_API_KEY:
            primary = AnthropicLLM(model=MODEL_ADVISORY, api_key=ANTHROPIC_API_KEY)
            return FallbackLLM(primary=primary, fallbacks=[sub_llm, fast_llm] + openai_fallback)
        # Anthropic unavailable - fall through to subscriber routing

    if role in ("free", "student"):
        if openai_fallback:
            return FallbackLLM(primary=fast_llm, fallbacks=openai_fallback)
        return fast_llm

    # subscriber / professional
    return FallbackLLM(primary=sub_llm, fallbacks=[fast_llm] + openai_fallback)
```

Updated Fallback Table

Role	Primary	Fallback 1	Fallback 2	Fallback 3
free, student	Groq llama-3.1-8b-instant	gpt-4o-mini	—	—
subscriber, professional	Groq llama-3.3-70b-versatile	Groq llama-3.1-8b-instant	gpt-4o-mini	—
advisory, admin	Claude Sonnet 4.6 (Anthropic)	Groq llama-3.3-70b-versatile	Groq llama-3.1-8b-instant	gpt-4o-mini

Behavior: `FallbackLLM._is_retryable(exc)` determines whether to try the next model. Only transient errors (429 rate limit, 500/502/503/504 server errors, "overloaded", "timeout") trigger fallback. Validation errors (400 bad request) are re-raised immediately — fallback would not help.

OpenAI installation requirement:

```
pip install llama-index-llms-openai
```

This package must be included in `backend/requirements.txt`. The code uses a try/except guard so that if the package is missing, the OpenAI fallback is silently omitted rather than crashing the backend.

9. HTR Connect Hub

What Changed

`/connect` is a new landing page for HTR's community and peer learning features. It was added to the left sidebar navigation under the **Advisory** section.

Page Architecture

`frontend/app/connect/page.tsx` (New — Server Component)

A static grid of 8 feature cards, each linking to a community sub-feature:

Feature	Route	Description
Ask HTR	<code>/connect</code>	Expert Q&A with 48-hour response guarantee
Pillar Circles	<code>/connect</code>	Moderated discussion forums, one per intelligence pillar

Feature	Route	Description
Peer Cohorts	/connect	Quarterly learning groups by org type (10–25 members)
Member Directory	/connect/directory	Opt-in searchable professional directory
Toolkits & Templates	/connect	Downloadable practitioner frameworks
Office Hours	/connect	Monthly group sessions with HTR analysts
Policy Alerts	/connect	Real-time CMS/state regulatory notifications
Cohort Application	/connect	Rolling admissions portal

The page uses the standard `HubPageTemplate` layout for visual consistency with other hub pages (Policy, Economics, Technology, etc.).

Navigation

`frontend/components/HomeSidebar.tsx` (Modified)

Added `/connect` and `/connect/directory` to the **Advisory** section:

```
advisoryItems = [
  ...existing items...,
  { href: "/connect", label: "HTR Connect", icon: UserGroupIcon },
  { href: "/connect/directory", label: "Member Directory", icon: IdentificationIcon },
]
```

`advisoryPrefixes` updated to include `/connect` so the Advisory section auto-expands when on any Connect page.

10. Professional Member Directory

Overview

A subscriber-only opt-in directory allowing healthcare professionals to create a public profile and search peers by organization type, pillar interest, and state.

Database Schema

professional_profiles table (created in Migration 023):

```
CREATE TABLE professional_profiles (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL UNIQUE REFERENCES auth.users(id) ON DELETE CASCADE,
  display_name TEXT NOT NULL CHECK (char_length(display_name) BETWEEN 2 AND 80),
  title       TEXT CHECK (char_length(title) <= 120),
  organization TEXT CHECK (char_length(organization) <= 120),
  organization_type TEXT CHECK (organization_type IN (
    'health_system', 'health_plan', 'state_agency', 'federal_agency',
    'aco', 'fqhc', 'cah', 'amc', 'consulting', 'vendor', 'nonprofit', 'academia', 'other'
  )),
  bio         TEXT CHECK (char_length(bio) <= 600),
  linkedin_url TEXT CHECK (char_length(linkedin_url) <= 300),
  pillars     TEXT[] DEFAULT '{}',
  state       TEXT,
  is_public   BOOLEAN NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

RLS policies:

- `profiles_public_read` — `SELECT` allowed for any authenticated user where `is_public = TRUE`
- `profiles_own_read` — `SELECT` allowed for a user's own profile regardless of `is_public`
- `profiles_own_write` — `INSERT`, `UPDATE`, `DELETE` only where `user_id = auth.uid()`

This means users can browse public profiles, manage their own profile, and set `is_public = FALSE` to hide from search while retaining their data.

API Route

`frontend/app/api/directory/route.ts` (New)

GET — search public profiles:

```
GET /api/directory?q=cardiology&org_type=health_system&pillar=clinical&state=vermont

Filters applied (all optional, combined with AND):
- q:      OR(ilike display_name, ilike organization, ilike title)
```

```
- org_type: eq("organization_type", value)
- pillar:  contains("pillars", [value])  - array containment
- state:   eq("state", value)

Order: display_name ASC
Limit: 100
Requires: authenticated session
```

POST — upsert own profile:

```
POST /api/directory
Body: {
  display_name:      string (required, trimmed to 80 chars),
  title?:           string,
  organization?:     string,
  organization_type?:string,
  bio?:             string,
  pillars?:         string[],
  state?:           string,
  linkedin_url?:    string,
  is_public?:       boolean
}

Upsert conflict: user_id (each user has exactly one profile)
Requires: authenticated session
```

Frontend

frontend/app/connect/directory/page.tsx (New — Server Component)

- Authentication guard: redirects to `/login?from=/connect/directory` if unauthenticated
- Wraps `<DirectoryClient />` in `<Suspense>` with a skeleton fallback

frontend/app/connect/directory/DirectoryClient.tsx (New — Client Component)

Profile search:

- Text search input (debounced 300ms)
- Org type dropdown (13 types with human-readable labels)
- Pillar dropdown (Policy, Economics, Technology, Clinical, Equity)
- Results rendered as a 3-column card grid on large screens

Profile card:

```
[Display Name]
[Title] at [Organization]
[Org Type Badge]

[Bio excerpt – first 150 chars]

[Pillar tags with color coding]
  policy: sky | economics: emerald | technology: indigo
  clinical: rose | equity: orange
[LinkedIn →] if provided
```

Edit modal:

- Full form with all profile fields
- `is_public` toggle (shows/hides from directory)
- Pillar checkboxes (multi-select)
- Saves via `POST /api/directory`
- Opens automatically on first visit if user has no profile yet

Organization type labels:

DB value	Display label
health_system	Health System
health_plan	Health Plan / Payer
state_agency	State Agency
federal_agency	Federal Agency
aco	ACO
fqhc	FQHC
cah	Critical Access Hospital

DB value	Display label
amc	Academic Medical Center
consulting	Consulting / Advisory
vendor	Technology Vendor
nonprofit	Nonprofit / Foundation
academia	Academia / Research
other	Other

11. Research Lab — Bundle Optimization & Error Boundaries

Background

The Research Lab page (`/research-lab`) loaded all 19 interactive tools in a single bundle (~1.8MB gzipped). Every user paid the cost of loading all tools even if they only used one. Additionally, calculation errors in any tool silently crashed the entire Research Lab page.

Bundle Optimization

`frontend/app/research-lab/ResearchLabHub.tsx` (Modified)

All 19 tools are now dynamically imported with:

- `ssr: false` — tools use browser APIs (canvas, ResizeObserver), so they cannot render on the server
- `loading: () => <ToolSkeleton />` — animated placeholder shown while the chunk loads
- `/* webpackChunkName: "tool-xxx" */` magic comment — groups related tools into shared chunks to maximize vendor code reuse

```
const FHIRLab = dynamic(  
  () => import(/* webpackChunkName: "tool-fhir" */ '@components/research/FHIRLab'),  
  { ssr: false, loading: () => <ToolSkeleton /> }  
)  
  
const APMDesignLab = dynamic(  
  () => import(/* webpackChunkName: "tool-apm" */ '@components/research/APMDesignLab'),  
  { ssr: false, loading: () => <ToolSkeleton /> }  
)  
// ... all 19 tools
```

ToolSkeleton component — provides visual continuity while tools load:

```
function ToolSkeleton() {  
  return (  
    <div className="w-full animate-pulse space-y-4 p-6">  
      <div className="h-6 bg-slate-100 rounded-lg w-1/3" />  
      <div className="h-4 bg-slate-100 rounded w-2/3" />  
      <div className="grid grid-cols-3 gap-4 mt-6">  
        {[1, 2, 3].map(i => (  
          <div key={i} className="h-28 bg-slate-100 rounded-xl" />  
        ))}  
      </div>  
      <div className="h-48 bg-slate-100 rounded-2xl mt-4" />  
    </div>  
  )  
}
```

Chunk grouping strategy:

Chunk name	Tools included	Why grouped
tool-fhir	FHIRLab	Standalone (large, specialized)
tool-risk	RiskStratificationEngine	Standalone
tool-apm	APMDesignLab, APMCalculator	Share payment model logic
tool-econ	CEACalculator	Economics-specific
tool-pop	PopulationHealthModeler	Standalone (maps)
tool-equity	HealthEquityStudio	Standalone
tool-policy	PolicySimulator	Standalone
tool-clinical	ClinicalQualityOptimizer	Standalone
tool-finance	HospitalFinancialScorecard	Standalone
tool-hta	HTAStudio, ActuarialLab	Share health economics libs

Chunk name	Tools included	Why grouped
tool-ai	AIAnalyticsLab, DigitalHealthLab	Share AI/ML dependencies
tool-knowledge	EvidenceLibrary, WorkforceModeler, InnovationLeaderboard, ResearchWorkspace	Light tools, shared bundle

Error Boundaries

frontend/components/ErrorMessage.tsx (Referenced — already existed)

Each tool is now wrapped in <ErrorMessage key={activeToolId}>:

```
<ErrorMessage key={`_${activeSection}-${activeTool}`}>
  <Suspense fallback={<ToolSkeleton />}>
    <ActiveToolComponent />
  </Suspense>
</ErrorMessage>
```

The **key** prop is critical: when the user switches tools, React unmounts the old **ErrorMessage** and mounts a fresh one. This means an error in Tool A does not permanently disable Tool B. It also allows the user to recover from an error by switching away and back.

ErrorMessage behavior:

- Catches all rendering and lifecycle errors within the tool
- Displays a "Something went wrong" message with a "Reset Tool" button
- Reports error to Sentry with the **toolId** as context
- "Reset Tool" button calls **this.setState({ hasError: false })** — remounts the tool

Impact

Metric	Before	After
Initial JS bundle (research-lab)	~1.8MB gzipped	~180KB (index + tab UI only)
First meaningful paint	Blocked on all 19 tools	Immediate — tools load on demand
Tool crash isolation	One crash = full page broken	Crash contained to tool card

12. Admin Revenue Dashboard

What Changed

A new **Revenue** admin page is available at **/admin/revenue**, linked from the main admin dashboard. It provides real-time MRR, churn rate, trial conversion, and subscription breakdowns without requiring access to the Stripe dashboard.

Architecture

frontend/app/admin/revenue/page.tsx (New — Server Component)

```
requireRole("admin")
|
|
▼
getRevenueData() — fetches all rows from subscriptions table
├── active:    filter status IN ('active', 'trialing')
├── trials:    filter status = 'trialing'
├── churned:   filter status = 'canceled' AND created_at > NOW()-30d
├── planCounts: group by plan for active subscriptions
├── mrr:       SUM(count * PLAN_PRICES[plan]) for active plans
├── conversionRate: activeNonTrial / totalEver * 100 (all-time estimate)
└── recent:    last 20 rows ordered by created_at DESC
```

Plan pricing constants:

```
const PLAN_PRICES: Record<string, number> = {
  subscriber: 29,
  student:    19,
  professional: 99,
  advisory:   299,
};
```

Metrics displayed:

Metric	Calculation	Notes
MRR	$\sum (\text{plan_count} \times \text{plan_price})$	Active + trialing only

Metric	Calculation	Notes
ARR (est.)	MRR × 12	Shown below plan breakdown
Active	COUNT where status IN ('active','trialing')	
Trialing	COUNT where status = 'trialing'	Subset of Active
Churned (30d)	COUNT where status = 'canceled' AND recent	
Trial→Paid	active_non_trial / total_ever_subscribed	All-time estimate

Status badge colors:

Status	Color
active	Emerald
trialing	Blue
past_due	Amber
canceled	Slate
paused	Slate

Admin Dashboard Link

frontend/app/admin/page.tsx (Modified) — "Revenue Dashboard" link added to the Admin Tools quick-links section, pointing to `/admin/revenue`.

13. Hospital Real Data Pipeline

Background

The State Detail page's Hospital tab displayed synthetic data. This change provides:

1. A script to download and load real CMS hospital data into Supabase
2. A fallback chain in the dashboard page: Sanity CMS → Supabase (CMS data) → empty

CMS Data Loader Script

backend/scripts/load_cms_hospitals.py (New)

Downloads the CMS Provider of Services (POS) flat file from **data.cms.gov** and upserts into the **hospitals** Supabase table.

Data source:

```
URL: https://data.cms.gov/resource/fc9f-b161.csv
    ?$limit=50000
    &$select=prvdr_num,fac_name,city_name,state_cd,prvdr_ctgry_cd,bed_cnt,pgm_trmtn_cd
```

Hospital type mapping:

CMS Category Code	Hospital Type	Notes
01	Urban	Short-term acute care
11	Critical Access	CAH designation
14	Rural PPS	Rural prospective payment

Termination filtering: Providers with **pgm_trmtn_cd** set to anything other than blank or **"00"** are excluded — these are closed or terminated facilities.

Discharge estimation (when real discharge data is unavailable):

```
estimated_discharges = int(beds * 0.60 * 365 / 4)
# Assumptions: 60% occupancy, 4-day average length of stay
```

This is explicitly labeled as an estimate. When CMS publishes actual discharge counts (available in the separate Hospital Compare dataset), those values should replace this estimate via a follow-up script.

Running the loader:

```
# Full load of all states
cd backend
source venv/bin/activate
python -m scripts.load_cms_hospitals
```

```
# Single state (faster for testing)
python -m scripts.load_cms_hospitals --state VT

# Dry run - prints records without writing
python -m scripts.load_cms_hospitals --state VT --dry-run
```

Scheduling: Run quarterly after CMS releases updated POS files (typically January, April, July, October). Consider adding a Railway cron job:

```
0 6 15 1,4,7,10 * → python -m scripts.load_cms_hospitals
```

Dashboard Fallback Chain

`frontend/app/dashboard/[state]/page.tsx` (Modified)

The page now fetches from both Sanity and Supabase in parallel:

```
const [sanityIndex, sanityProgram, sanityHospitals, dbMetrics, dbProfile, dbHospitals] =
  await Promise.all([
    getPerformanceIndex(stateSlug),
    getRhtState(stateSlug),
    getHospitalsByState(stateSlug),    // Sanity CMS
    getStateMetrics(stateSlug),
    getRhtProfile(stateSlug),
    getHospitals(stateSlug),          // Supabase (CMS-loaded data)
  ]);
```

Hospital data priority:

```
const hospitals = sanityHospitals.length > 0
  ? sanityHospitals                // Prefer Sanity (editor-curated)
  : dbHospitals.map(h => ({        // Fall back to CMS POS data
    id:          h.id,
    name:        h.name,
    city:        h.city,
    type:        h.hospital_type as "Critical Access" | "Rural PPS" | "Urban",
    totalDischarges: h.total_discharges,
    avgLengthOfStay: h.avg_length_of_stay,
    qualityScore:  h.quality_score,
  }));
```

Full priority hierarchy for all data on the state detail page:

Data type	Priority 1	Priority 2	Priority 3
Performance Index	Sanity CMS	Supabase <code>state_health_metrics</code>	Static JS file
RHT Profile	Sanity CMS	Supabase <code>rht_profiles</code>	Static JS file
Hospitals	Sanity CMS	Supabase <code>hospitals</code> (CMS data)	Empty

14. Database Migration 023

File: `supabase/migrations/023_api_key_rotation_and_directory.sql`

This migration adds two capabilities: API key rotation metadata and the professional directory.

API Keys — Schema Extensions

```
-- Add rotation fields to existing api_keys table
ALTER TABLE api_keys
  ADD COLUMN IF NOT EXISTS expires_at    TIMESTAMPTZ,
  ADD COLUMN IF NOT EXISTS rotated_from  UUID REFERENCES api_keys(id) ON DELETE SET NULL;

-- Partial index - only indexes keys that have an expiry and aren't yet revoked
-- Keeps the index small and the cleanup query fast
CREATE INDEX IF NOT EXISTS api_keys_expires_idx
  ON api_keys(expires_at)
  WHERE expires_at IS NOT NULL AND revoked_at IS NULL;
```

Automated cleanup function:

```
CREATE OR REPLACE FUNCTION revoke_expired_api_keys()
RETURNS void LANGUAGE plpgsql SECURITY DEFINER AS $$
```

```
BEGIN
  UPDATE api_keys
  SET     revoked_at = NOW()
  WHERE  expires_at < NOW()
  AND    revoked_at IS NULL;
END;
$$;
```

To schedule via pg_cron (run in Supabase SQL Editor after enabling pg_cron):

```
SELECT cron.schedule(
  'revoke-expired-keys',
  '0 * * * *', -- hourly
  'SELECT revoke_expired_api_keys();'
);
```

Professional Directory — New Table

```
CREATE TABLE professional_profiles (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID NOT NULL UNIQUE REFERENCES auth.users(id) ON DELETE CASCADE,
  display_name      TEXT NOT NULL CHECK (char_length(display_name) BETWEEN 2 AND 80),
  title             TEXT CHECK (char_length(title) <= 120),
  organization       TEXT CHECK (char_length(organization) <= 120),
  organization_type TEXT CHECK (organization_type IN (
    'health_system', 'health_plan', 'state_agency', 'federal_agency',
    'aco', 'fqhc', 'cah', 'amc', 'consulting', 'vendor', 'nonprofit', 'academia', 'other'
  )),
  bio               TEXT CHECK (char_length(bio) <= 600),
  linkedin_url      TEXT CHECK (char_length(linkedin_url) <= 300),
  pillars           TEXT[] DEFAULT '{}',
  state             TEXT,
  is_public         BOOLEAN NOT NULL DEFAULT TRUE,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- RLS
ALTER TABLE professional_profiles ENABLE ROW LEVEL SECURITY;

CREATE POLICY "profiles_public_read" ON professional_profiles
  FOR SELECT USING (is_public = TRUE AND auth.uid() IS NOT NULL);

CREATE POLICY "profiles_own_read" ON professional_profiles
  FOR SELECT USING (auth.uid() = user_id);

CREATE POLICY "profiles_own_write" ON professional_profiles
  FOR ALL USING (auth.uid() = user_id);

-- Automatic updated_at
CREATE TRIGGER set_professional_profiles_updated_at
  BEFORE UPDATE ON professional_profiles
  FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
```

Applying this migration:

```
supabase db push
# or
supabase migration up
```

15. Environment Variables — New Additions

Backend ([backend/.env](#))

Variable	Required	Description
OPENAI_API_KEY	Recommended	Powers OpenAI gpt-4o-mini last-resort fallback. Without it, fallback chain terminates at Groq. Also required for OpenAI embeddings (text-embedding-3-small).

[OPENAI_API_KEY](#) was already used for embeddings — this change adds its use in the LLM fallback chain. No new variable required, but ensure it is set in production.

Frontend ([frontend/.env.local](#) / Vercel)

No new variables required for this amendment. Variables added in v4.3.0 (`API_KEY_HMAC_SECRET`, `ANTHROPIC_API_KEY`, `LOOPS_API_KEY`, `NEXT_PUBLIC_APP_URL`) remain required.

Sanity Webhook

Setting	Value
Webhook URL	<code>https://<your-domain>/api/webhooks/sanity</code>
Secret	Same as <code>INGEST_SECRET</code> in <code>backend/.env</code>

The `INGEST_SECRET` variable was already required for the manual `/api/ingest` endpoint. No new secret needed.

16. Updated Known Issues Register

Items marked  **Resolved** were open in the v4.3.0 register.

Issue	Severity	Status	Notes
Right sidebar send button hidden on short viewports	High	 Resolved	Fixed: <code>minHeight: "30vh" + maxHeight: calc(100vh - 7rem)</code> on <code>CollapsibleSidebar</code> wrapper. Sidebar grows before scrolling.
Academy enrollment redirect post-payment broken	High	 Resolved	<code>EnrollRedirect</code> client component reads <code>from</code> param on <code>/account/subscription?success=true&from=...</code> and redirects after 2 seconds.
Bookmarks list on <code>/account</code> shows placeholder	Medium	 Resolved	<code>BookmarksList.tsx</code> renders real bookmarks from Supabase with delete functionality.
Personalized learning path not persisting	Medium	 Resolved	<code>PersonalizedLearningHub</code> uses <code>localStorage</code> + Supabase <code>user_learning_paths</code> table with load-on-mount logic.
HTI Dashboard uses static mock data	Medium	 Resolved	<code>/api/hti-scores</code> wired to Supabase <code>hti_scores</code> . Amber overlay shown when real data exists.
Hospital View tab uses synthetic data	Medium	 Resolved	CMS POS loader script provided. Dashboard falls back to <code>hospitals</code> Supabase table.
Research Lab tools have no error boundaries	Low	 Resolved	All 19 tools wrapped in <code><ErrorBoundary key={toolId}></code> . Crashes isolated per tool.
<code>/community</code> route is a placeholder	Low	 Resolved	Replaced by HTR Connect hub at <code>/connect</code> with 8 feature cards and live directory at <code>/connect/directory</code> .
<code>/investment-tracker</code> route does not exist	Low	 Resolved	Implemented in v4.3.0.
Mobile sidebar close requires two taps	High	Open	z-index / event propagation issue on mobile. Planned for v4.5.0.
FlashRank model download on first startup	Low	Acceptable	One-time ~100MB download; cached at <code>/tmp/flashrank</code> after. Railway volumes retain cache across deploys.
In-process rate limiter resets on cold start	Low	Acceptable	Affects only <code>/verify/</code> — low traffic, acceptable for current scale.
Hospital discharge counts are estimates	Medium	Open	<code>load_cms_hospitals.py</code> estimates from bed count. CMS Hospital Compare dataset provides actual counts — follow-up script planned.
<code>quality_score</code> in hospital data is placeholder	Medium	Open	Set to 75 uniformly. HCAHPS star ratings from CMS Hospital Compare would replace this.

End of Platform Amendment v4.4.0. This document supersedes the open-issue register in v4.3.0 for all items listed above. All prior sections of v4.3.0 and the printed v4.2.0 documentation remain valid and in effect.